

Projet : Young Speaker — Plateforme d'expression des jeunes

1. Vision du projet

Créer une plateforme web moderne appelée **Young Speaker**, destinée principalement aux adolescents et aux jeunes.

Young Speaker est un espace sécurisé où les jeunes peuvent **s'exprimer librement sur des sujets qui les concernent**, notamment :

- Santé sexuelle et reproductive
- Santé mentale et bien-être
- Relations amoureuses et amicales
- Puberté et adolescence
- Vie scolaire et pression sociale
- Harcèlement
- Réseaux sociaux
- Confiance en soi
- Éducation
- Addictions
- Violences
- Genre et égalité
- Vie familiale
- Orientation professionnelle
- Développement personnel
- Citoyenneté
- Autres sujets liés à la jeunesse

L'objectif n'est pas uniquement de publier des articles, mais de créer une **communauté d'expression, d'écoute, de sensibilisation et de participation des jeunes**.

La plateforme doit permettre à un jeune de devenir **Young Speaker**, de publier son expérience, son opinion, son témoignage ou son analyse sur un thème donné.

D'autres visiteurs pourront lire les publications, réagir et voter.

2. Concept principal

La plateforme doit fonctionner autour de 5 piliers :

A. Young Speakers

Un jeune peut s'inscrire comme **Young Speaker**.

Il possède un profil public avec :

- Nom ou pseudonyme
- Photo/avatar
- Âge ou tranche d'âge
- Ville/pays
- Courte biographie
- Centres d'intérêt
- Nombre d'articles publiés
- Nombre de votes reçus
- Badges obtenus

Le jeune peut proposer des articles.

Une publication peut être :

- Opinion
- Témoignage
- Article
- Expérience personnelle
- Réflexion
- Conseils
- Interview
- Poème / texte libre
- Vidéo ou contenu multimédia dans une évolution future

B. Thème de la semaine

Chaque semaine, l'administration définit un **Thème de la semaine**.

Exemple :

Thème de la semaine :

"Pourquoi est-il difficile pour les jeunes de parler de leur santé mentale ?"

Les Young Speakers peuvent publier autour de ce thème.

La page d'accueil doit mettre fortement en avant le thème actuel.

Afficher également :

- Date de début
- Date de fin
- Nombre de participations
- Nombre de votes
- Articles participants

Prévoir également les thèmes à venir.

C. Vote de la communauté

Les visiteurs peuvent voter pour les articles qu'ils apprécient.

Chaque semaine, les articles participants sont classés selon les votes.

Afficher :

Top Young Speakers de la semaine

1. 🏆 Premier
2. 🥈 Deuxième
3. 🥉 Troisième

Afficher également :

- Nombre de votes
- Nombre de lectures
- Nombre de commentaires
- Nom/pseudonyme du Young Speaker

Le système doit empêcher autant que possible les votes abusifs.

Prévoir une limitation des votes par utilisateur/session/IP/device selon l'architecture retenue.

Le système de vote devra être conçu pour pouvoir évoluer vers une authentification obligatoire si nécessaire.

3. Challenges

Créer une section **Challenges**.

Les challenges sont proposés par l'administration et permettent aux jeunes de participer à des activités de sensibilisation, de réflexion ou de développement personnel.

Exemples :

Challenge 1

"Parle à une personne de confiance cette semaine."

Challenge 2

"Écris 5 choses que tu apprécies chez toi."

Challenge 3

"Déconstruis une fausse information sur la santé sexuelle."

Challenge 4

"7 jours sans jugement."

Chaque challenge possède :

- Titre
- Description
- Objectif
- Image
- Date de début
- Date de fin
- Niveau de difficulté
- Nombre de participants
- Instructions
- Récompense éventuelle
- Statut

Les jeunes peuvent :

- Participer
- Marquer le challenge comme terminé
- Partager leur expérience
- Obtenir des points/badges

4. Système de badges

Créer un système de gamification simple.

Exemples :

- 🎤 Young Speaker
- 📝 Premier article
- 🔥 Speaker actif
- 🏆 Article de la semaine
- 💬 Voice of Youth
- 🌟 Community Contributor
- 🧠 Mental Health Advocate
- ❤️ SRHR Advocate
- 🏅 Challenge Master

Les badges doivent pouvoir être configurés depuis l'administration.

5. Architecture technique

Utiliser :

Frontend + Backend

Next.js avec App Router

Utiliser TypeScript.

Architecture full-stack Next.js.

Prévoir :

- Server Components lorsque pertinent
 - Server Actions lorsque pertinent
 - API Routes / Route Handlers lorsque nécessaire
 - Authentification
 - Middleware
 - Validation côté serveur
 - Validation côté client
 - Gestion des erreurs
 - Protection des routes
-

6. Stack recommandée

Utiliser :

- Next.js récent
- TypeScript
- Tailwind CSS
- shadcn/ui
- Prisma ORM
- PostgreSQL
- Auth.js / solution d'authentification adaptée
- Zod pour la validation
- React Hook Form pour les formulaires
- Lucide Icons

Préparer l'application pour pouvoir intégrer plus tard :

- Cloudinary / UploadThing / S3 pour les médias
- Redis
- Elasticsearch / moteur de recherche
- Notifications
- Analytics
- PWA / application mobile

Ne pas ajouter inutilement ces services dans le MVP.

7. Types d'utilisateurs

Créer au minimum les rôles suivants :

VISITOR

Visiteur non connecté.

Peut :

- Consulter les articles
- Rechercher
- Consulter les profils publics
- Voir les challenges
- Voir le thème de la semaine
- Voter selon les règles définies

YOUNG_SPEAKER

Peut :

- Créer son profil
- Publier un article
- Modifier ses brouillons
- Voir ses publications
- Voir ses statistiques
- Participer aux challenges
- Obtenir des badges
- Commenter
- Voter

MODERATOR

Peut :

- Modérer les articles
- Modérer les commentaires
- Signaler/suspendre des contenus
- Valider ou rejeter des publications
- Gérer les signalements

ADMIN

Peut :

- Gérer les utilisateurs
- Gérer les Young Speakers
- Gérer les articles
- Gérer les thèmes
- Gérer les challenges

- Gérer les votes
- Gérer les commentaires
- Gérer les catégories
- Gérer les badges
- Gérer les signalements
- Consulter les statistiques
- Gérer les paramètres de la plateforme

8. Modération — élément essentiel

Les sujets traités peuvent être sensibles.

La plateforme doit donc intégrer un système de modération.

Chaque article possède un statut :

DRAFT
PENDING_REVIEW
APPROVED
REJECTED
ARCHIVED

Lorsqu'un Young Speaker soumet un article :

DRAFT
↓
PENDING_REVIEW
↓
MODERATION
↓
APPROVED
↓
PUBLIC

L'administrateur/modérateur peut :

- Approuver
- Rejeter
- Demander une modification
- Masquer
- Archiver

Lorsqu'un article est rejeté, le Young Speaker doit pouvoir recevoir une raison du rejet.

9. Avertissement sur les contenus sensibles

Certains articles peuvent concerner :

- Santé mentale
- Violences
- Santé sexuelle
- Traumatismes
- Addictions
- Sujets médicaux

Prévoir la possibilité pour l'administration d'ajouter un avertissement de contenu.

Exemple :

⚠ Ce contenu aborde des sujets sensibles liés à la santé mentale.

La plateforme ne doit pas présenter les articles comme des conseils médicaux professionnels.

Prévoir également un système permettant d'afficher des ressources d'aide lorsque cela est pertinent.

10. Articles

Créer un éditeur d'article moderne.

Un article doit contenir :

```
id
authorId
title
slug
excerpt
content
coverImage
categoryId
weeklyThemeId
status
views
voteCount
commentCount
createdAt
updatedAt
publishedAt
```


Fonctionnalités :

- Titre
- Image de couverture
- Extrait
- Contenu riche
- Catégorie
- Thème de la semaine
- Brouillon automatique si possible
- Prévisualisation
- Publication
- Modification
- Suppression
- Signalement

Créer des URLs SEO-friendly :

/articles/pourquoi-les-jeunes-parlent-peu-de-sante-mentale

11. Catégories

Créer des catégories administrables.

Exemples :

- Santé mentale
- Santé sexuelle et reproductive
- Relations
- Adolescence
- Éducation
- Société
- Développement personnel
- Témoignages
- Jeunesse
- Autres

12. Page d'accueil

Créer une landing page très moderne et orientée jeunesse.

Structure :

Hero

Titre :

Ta voix compte. Fais-la entendre.

Sous-titre :

Young Speaker est l'espace où les jeunes partagent leurs idées, leurs expériences et leurs réalités.

CTA :

Devenir Young Speaker

CTA secondaire :

Découvrir les voix des jeunes

Thème de la semaine

Grande section :

 THÈME DE LA SEMAINE

Afficher le thème actuel.

CTA :

Participer au thème

Articles populaires

Afficher les meilleurs articles.

Cards avec :

- Image
 - Titre
 - Auteur
 - Catégorie
 - Nombre de votes
 - Nombre de vues
 - Date
-

Young Speakers à découvrir

Afficher les profils les plus actifs.

Challenge actuel

Présenter le challenge de la semaine.

Classement

Afficher :

 Young Speakers de la semaine

Pourquoi devenir Young Speaker ?

Expliquer :

- Faire entendre sa voix
 - Partager son expérience
 - Sensibiliser d'autres jeunes
 - Développer ses compétences
 - Participer aux challenges
 - Construire son profil
-

13. Page "Devenir Young Speaker"

Créer une page dédiée.

Expliquer clairement le concept.

Processus :

1. Crée ton compte
↓
2. Complète ton profil
↓
3. Choisis un thème
↓
4. Écris ton article
↓
5. Soumets-le
↓
6. Modération
↓
7. Publication
↓
8. La communauté vote

CTA :

Je deviens Young Speaker

14. Dashboard Young Speaker

Créer un dashboard.

Afficher :

- Nombre d'articles
- Articles publiés
- Articles en attente
- Nombre total de vues
- Nombre total de votes
- Classement
- Badges
- Challenges
- Notifications

Menu :

Dashboard
Mes articles
Nouvel article
Mes statistiques
Mes challenges
Mes badges
Mon profil
Paramètres
Déconnexion

15. Dashboard administration

Créer un dashboard admin professionnel.

Statistiques :

- Nombre d'utilisateurs
- Nombre de Young Speakers
- Nombre d'articles
- Articles en attente
- Nombre de votes
- Nombre de commentaires
- Nombre de challenges

- Nombre de participants
- Articles publiés cette semaine

Dashboard avec graphiques.

Sections :

Dashboard
Utilisateurs
Young Speakers
Articles
Modération
Thèmes
Challenges
Catégories
Badges
Commentaires
Signalements
Votes
Statistiques
Paramètres

16. Commentaires

Permettre aux utilisateurs connectés de commenter les articles.

Prévoir :

- Commentaire
- Réponse à un commentaire
- Suppression
- Signalement
- Modération

Éviter les commentaires anonymes pour faciliter la modération.

17. Signalement

Sur chaque article/commentaire :

Signaler

Motifs :

- Contenu offensant

- Harcèlement
- Désinformation
- Contenu dangereux
- Contenu sexuel inapproprié
- Discours haineux
- Spam
- Autre

Créer une table de signalements.

18. Recherche

Créer une recherche globale.

Recherche sur :

- Articles
- Young Speakers
- Challenges
- Thèmes

Prévoir une architecture permettant de remplacer facilement la recherche PostgreSQL par Elasticsearch plus tard.

19. SEO

Le projet doit être SEO-friendly.

Prévoir :

- Metadata dynamique
- Open Graph
- Twitter/X cards
- Sitemap
- Robots.txt
- URLs propres
- Slugs
- Structured data lorsque pertinent
- Pages indexables
- Images optimisées
- `generateMetadata`

Chaque article doit avoir ses propres metadata.

20. Responsive design

La plateforme doit être conçue **mobile-first**.

Priorité :

1. Mobile
2. Tablet
3. Desktop

L'expérience mobile doit être excellente car le public principal est jeune.

21. Design

Créer une identité visuelle moderne, jeune, inclusive et rassurante.

Éviter un design trop institutionnel.

Le design doit communiquer :

- Jeunesse
- Expression
- Confiance
- Écoute
- Sécurité
- Diversité
- Engagement

Utiliser des cartes modernes, des arrondis, des animations légères et une bonne hiérarchie visuelle.

Prévoir :

- Light mode
 - Dark mode
 - Accessibilité
 - Contrastes suffisants
 - Navigation clavier
 - États loading
 - Skeleton loaders
 - Empty states
 - Error states
-

22. Base de données

Créer un schéma Prisma propre et normalisé.

Prévoir au minimum les modèles :

```
User
Profile
Article
Category
WeeklyTheme
Vote
Comment
Challenge
ChallengeParticipation
Badge
UserBadge
Report
Notification
```

Relations correctement définies.

Ajouter :

- indexes
- unique constraints
- timestamps
- soft delete lorsque pertinent

Les statistiques comme `voteCount`, `views` et `commentCount` peuvent être dénormalisées pour les performances, mais maintenir leur cohérence.

23. Sécurité

La sécurité est importante.

Implémenter :

- Hash sécurisé des mots de passe si authentification par mot de passe
- Protection CSRF selon architecture
- Rate limiting
- Validation Zod
- Sanitization du contenu riche
- Protection XSS
- Protection contre les votes abusifs
- Authorization côté serveur
- RBAC
- Vérification des permissions
- Protection des endpoints admin
- Logs des actions sensibles

Ne jamais faire confiance uniquement aux contrôles frontend.

Toutes les permissions doivent être vérifiées côté serveur.

24. Architecture du code

Créer une architecture propre et maintenable.

Exemple :

```
src/
├── app/
│   ├── (public)/
│   │   ├── page.tsx
│   │   ├── articles/
│   │   ├── speakers/
│   │   ├── challenges/
│   │   └── themes/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── register/
│   │   └── become-speaker/
│   ├── dashboard/
│   ├── admin/
│   └── api/
├── components/
│   ├── ui/
│   ├── articles/
│   ├── speakers/
│   ├── challenges/
│   ├── voting/
│   └── admin/
├── lib/
│   ├── auth/
│   ├── db/
│   ├── validations/
│   ├── permissions/
│   ├── rate-limit/
│   └── utils/
├── actions/
└── hooks/
```

|
└ types/

Adapter cette structure si une meilleure architecture Next.js est plus appropriée.

25. API / Server Actions

Créer des endpoints ou Server Actions propres.

Exemples :

```
POST /api/articles
GET /api/articles
GET /api/articles/[slug]
PATCH /api/articles/[id]
DELETE /api/articles/[id]

POST /api/articles/[id]/vote
DELETE /api/articles/[id]/vote

POST /api/articles/[id]/comments
GET /api/articles/[id]/comments

GET /api/themes/current
GET /api/themes
POST /api/themes

GET /api/challenges
POST /api/challenges/[id]/participate

GET /api/speakers
GET /api/speakers/[username]

POST /api/reports
```

Si Server Actions sont plus adaptées pour certaines opérations internes, les utiliser.

26. Notifications

Préparer le système de notification.

Exemples :

- Article approuvé
- Article rejeté

- Nouveau commentaire
- Article ayant reçu un vote
- Nouveau badge
- Nouveau challenge
- Nouveau thème

Architecture permettant d'ajouter plus tard :

- Email
 - Push notification
 - WhatsApp/SMS si nécessaire
-

27. MVP

Ne pas essayer de construire toutes les fonctionnalités avancées immédiatement.

Le MVP doit contenir :

Public

- Landing page
- Articles
- Article détail
- Thèmes
- Challenges
- Young Speakers
- Profils
- Recherche

Auth

- Inscription
- Connexion
- Profil
- Young Speaker

Young Speaker

- Dashboard
- Création d'article
- Brouillon
- Soumission
- Mes articles
- Statistiques basiques

Community

- Votes
- Commentaires
- Signalements

Admin

- Dashboard
 - Modération
 - Gestion articles
 - Gestion utilisateurs
 - Gestion thèmes
 - Gestion challenges
 - Gestion catégories
 - Gestion badges
-

28. Évolution future

Préparer l'architecture pour pouvoir ajouter plus tard :

- Application mobile
- PWA
- Podcasts
- Vidéos
- Lives
- Interviews
- Forum
- Messagerie
- Newsletter
- Notifications push
- IA pour recommandations de contenus
- IA pour aide à la rédaction
- Système de recommandation
- Traduction automatique français/anglais
- Système de récompenses
- Partenariats avec ONG et organisations de santé
- Statistiques avancées

L'IA ne doit pas être intégrée artificiellement dans le MVP. Préparer l'architecture afin qu'elle puisse être ajoutée proprement lorsque les données et les besoins le justifieront.

29. Données de démonstration

Créer un seed Prisma avec des données réalistes.

Créer :

- 10 Young Speakers
- 20 articles
- 5 catégories
- 4 thèmes hebdomadaires
- 5 challenges

- 10 badges
- Commentaires
- Votes
- Quelques signalements

Les données doivent permettre de voir immédiatement une plateforme vivante après installation.

30. Documentation

Créer un README complet contenant :

- Présentation du projet
- Stack
- Installation
- Variables d'environnement
- Configuration PostgreSQL
- Prisma
- Migration
- Seed
- Lancement en développement
- Build production
- Déploiement
- Structure du projet
- Comptes de démonstration
- Architecture
- Sécurité

Créer également un `.env.example`.

31. Docker

Préparer un environnement Docker pour le développement.

Créer :

```
Dockerfile
docker-compose.yml
.dockerignore
```

Le docker-compose doit pouvoir lancer au minimum :

```
Next.js
PostgreSQL
```

Préparer l'architecture pour Redis plus tard.

32. Qualité du code

Respecter :

- TypeScript strict
- ESLint
- Prettier
- Composants réutilisables
- Fonctions courtes
- Séparation des responsabilités
- Pas de duplication inutile
- Gestion propre des erreurs
- Types stricts
- Validation des données
- Accessibilité

Ne pas mettre toute la logique dans les composants React.

33. Important : ne pas faire un simple blog

Young Speaker doit être pensé comme une **plateforme communautaire participative jeunesse**.

Le cœur du produit est :

```
JEUNE
  ↓
DEVIENT YOUNG SPEAKER
  ↓
S'EXPRIME
  ↓
ARTICLE
  ↓
MODÉRATION
  ↓
PUBLICATION
  ↓
COMMUNAUTÉ
  ↓
LECTURES + COMMENTAIRES + VOTES
  ↓
CLASSEMENT
  ↓
RECONNAISSANCE
  ↓
CHALLENGES + BADGES
```

L'expérience utilisateur doit encourager les jeunes à **prendre la parole, écouter les autres et participer.**

34. Première mission de Codex

Avant d'écrire tout le code :

1. Analyser le projet.
2. Proposer l'architecture technique.
3. Proposer le schéma Prisma.
4. Identifier les éventuels problèmes de sécurité.
5. Définir les principales routes/pages.
6. Définir les rôles et permissions.
7. Définir le MVP.
8. Présenter brièvement le plan d'implémentation.

Puis commencer l'implémentation du MVP.

Ne pas demander de confirmation à chaque petite étape.

Construire progressivement une application fonctionnelle et cohérente.

À chaque étape importante :

- Vérifier TypeScript
- Vérifier ESLint
- Vérifier Prisma
- Vérifier les routes
- Vérifier les permissions
- Corriger les erreurs avant de continuer.

L'objectif final est d'obtenir une plateforme **Young Speaker fonctionnelle, moderne, sécurisée, responsive et prête à être déployée en production.**